

Snowflake Query Tagging Guidelines

- [Why do we need to tag queries in Snowflake?](#)
- [What is a query tag in Snowflake?](#)
- [Basic Example of Query Tagging](#)
- [Guidelines for Query Tagging at Chewy](#)
 - [JSON object Tags](#)
 - [Examples for different EDS/BI Applications](#)
 - [Python](#)
 - [Setting once upon connection creation](#)
 - [Modifying manually through alter session](#)
 - [Snowpark](#)
- [How to query Snowflake query_tag to understand cost & performance monitoring](#)

Why do we need to tag queries in Snowflake?

As we continue to mature our Snowflake offering at Chewy and migrate from Vertica, it is critical that we have the visibility needed to maintain optimal queries and schedules to attain optimal spend. Snowflake cost attribution is currently bundled at the Warehouse level. This doc sets standards that will provide Chewy with the ability to pinpoint exactly which applications/workflows are consuming credits.

Scenarios this aims to address:

1. Your manager tells you that you have to cut spend in Snowflake by 25% by the end of the year, and you aren't sure where there is the greatest opportunity to reduce cost.
2. What is the Snowflake cost of running a particular Tableau workbook? Are we getting the ROI?
3. Is there an automated workflow that keeps running long after the user left the company? Is this still needed?

Snowflake query tags allow users to associate arbitrary metadata with each query. The aim of standardized query tags is to achieve better visibility & monitoring for your Snowflake query costs and performance. Adding query tags exposes the information you need to get started.

What is a query tag in Snowflake?

Query tags are an optional [session level parameter](#) that allow users to tag any Snowflake SQL statement with a string. They can be up to 2000 characters in length, and contain any characters. Query tag values for each query can be found in the output of the `query_history` views in Snowflake, allowing them to be leveraged for a variety of use cases.

Query tags can also be used for more granular query performance monitoring. Users may wish to monitor the total runtime for each data model by grouping together the total elapsed time for all associated queries. Alternatively, if a set of queries are used to power user facing application dashboards, leveraging query tags can allow for more targeted performance monitoring.

Basic Example of Query Tagging

```
1  alter session set query_tag='users_model';
2
3  -- this query will be tagged with 'users_model'
4  create or replace table users_tmp as (
5      select *
6      from raw_users
7      where
8          not deleted
9          and created_at > current_date - 1
10 );
11
12 -- this will also be tagged with 'users_model'
13 insert into users
14 from users_tmp
15 ;
16
17 alter session set query_tag='orders_model';
18
19 -- this will get tagged with 'orders_model'
20 create or replace table orders as (
21     select *
22     from raw_orders
23     where
24         not deleted
25 );
```

 Query tagging happens at the session level. You may run the ALTER SESSION command prior to executing a query any number of times in a given session.

Guidelines for Query Tagging at Chewy

The `query_tag` allows for a string up to 2000 characters. Rather than using a simple string, we will be building a JSON object that contains most of the information we would

need to pinpoint the workflow.

JSON object Tags

Tag Name	Application	Required/Optional	Definition	Examples
owner	ANY/ALL	required	username of author or responsible party for the workflow	alattke, jmoore10, rderangula
app_name	ANY/ALL	required	Name of application running the query	tableau, knime, sandbox etl, snowflake scheduled task, my_awesome_app
workbook	Tableau	required	Name of the Tableau workbook	CS Dashboard
datasource	Tableau	required	Name of DataSource in given Tableau workbook	New Customer Count Query
dbt	DBT	required	package: get-select/dbt-query-tags	
statement_id	Python ETL	optional	Identifies specific query in a given workflow	1
file_path	Python Airflow KNIME Tableau	required (for python or .sql files)	file path and name that is being executed. Sandbox ETL would provide the relative path in GitHub to the file and Python should be the name of the python script filename.	/supply-chain-etl/blob/master/sql/supplychain_0500_a_item_attributes.sql , /supply-chain-etl/blob/master/scripts/my_awesome_app.py
comment	ANY	optional	Anything helpful related to the query	
dag	Airflow	required	DAG name	ollie.example.snowflake.dag
task	Airflow	required	Task name	inbound_transportation_collect_order_base_ps
data_filter_begin_dttm	Airflow Python ETL KNIME	optional	Start Datetime parameter supplied to script	20240118 00:00:00
data_filter_end_dttm	Airflow Python ETL	optional	End Datetime parameter supplied to script	20240130 13:59:35

- i The above are the standard set of tags. Only app_name and owner are required across the board. Others are required by application. You may choose to put other parameters that are relevant to your particular workflow.

Examples for different EDS/BI Applications

```

1  -- Tableau
2  alter session set query_tag = '{"owner": "alattke", "app_name": "tableau", "workbook":
3  "New Customer Count by Hour", "datasource": "Customer Count Query", "file_path": "New
4  Customer Count.twbx" }';
5
6  -- KNIME
7  alter session set query_tag = '{"owner": "mesperanza", "app_name": "Knime", "workflow":
8  "CS Customer Call Logs", "node": "Customer calls Query", "component": "Query #1"}';
9
10 -- Sandbox ETL
11 alter session set query_tag = '{"owner": "bdoshi", "app_name": "Sandbox ETL",
12 "file_path": "supply_chain_0500_uta_workflow.sql"}';

```

[Python/Airflow Example](#)

Python

If you are using Python to execute queries, there are two ways to set query tags.

Setting once upon connection creation

When you are creating your connection object with the [Snowflake Python Connector](#), you can set any session parameters upfront. In the example below, all queries executed from this `con` object will be tagged with the below JSON object that uniquely identifies the workflow .

```

1  con = snowflake.connector.connect(
2      user='rmanisha',
3      password='XXXX',
4      account='DEV_WH',
5      session_parameters={
6          'query_tag': '{"owner": "rmanisha",
7                      "application": "My Python Script",
8                      "filename": "my_python_script.py"
9                      }',
10     }
11 )

```

Modifying manually through `alter session`

If you don't want all queries in your session to have the same tag, you can instead run `alter session set query_tag = '{<my_JSON_object>}'` prior to running your actual queries. **This is the recommended way.**

```
1 import json
2
3 query_tag = {
4     "app_name": "data_lake_pipeline",
5     "owner": "pyahia",
6     "dag": "ollie.example.snowflake.dag",
7     "task": "inbound_transportation_collect_order_base_ps",
8     "data_filter_begin_dttm": "20240118 00:00:00",
9     "data_filter_end_dttm": "20240130 13:59:35",
10    "file_path": "days/ollie/examples/some-snowflake-file.sql",
11    "statement_id": 1
12 }
13 con.cursor().execute(f"alter session set query_tag='{json.dumps(query_tag)}'")
14
15 query = """
16 create or replace table users_tmp as (
17     select *
18     from raw_users
19     where 1=1
20         and not is_deleted is false
21         and created_at > current_date - 1
22 )
23 """
24
25 # runs the query and tags it with the supplied JSON string
26 con.cursor().execute(query)
27 con.cursor().execute("insert into users from users_tmp") # also tagged with same
28
29 # update the "statement_id" of the JSON object
30 query_tag['statement_id'] = 2;
31 con.cursor().execute(f"alter session set query_tag='{json.dumps(query_tag)}'")
32
33 query = """
34 create or replace table orders as (
35     select *
36     from raw_orders
37     where 1=1
38         and not is_deleted is false
39 )
40 """
41 # runs the second query and tags it with the supplied JSON string
42 con.cursor().execute(query)
```

For automated query tagging in Python, you can look at DSE's implementation (special thanks to @Phil Lynch):

Set up tags dict:

<https://github.com/Chewy-Inc/chewy-data-science-utils/blob/master/dsutils/snowflake.py#L467>

auto adds to init a cursor (or connection):

<https://github.com/Chewy-Inc/chewy-data-science-utils/blob/master/dsutils/snowflake.py#L33-L37>

[Snowpark](#)

How to query Snowflake `query_tag` to understand cost & performance monitoring

Query tags are surfaced in the [query history views](#) for each `query_id`. If the `query_tag` contains a JSON object, it can be parsed and segmented by any of the keys:

```
1 select
2     try_parse_json(query_tag)['app_name']::string as application_name,
3     count(*) as num_executions,
4     avg(total_elapsed_time/1000) as avg_total_elapsed_time_s
5 from sf_system_views.account_usage.query_history
6 where
7     try_parse_json(query_tag)['owner']::string = 'rmanisha'
8     and start_time > current_date - 30
9 group by 1
```